Bottom-up Syntax Analisys
ELR (*k*) Method – II

Translated and adapted by L. Breveglieri

CONDITION *ELR* (1) – makes determinism possible

PART 1 – no m-state has any

shift-reduce conflict

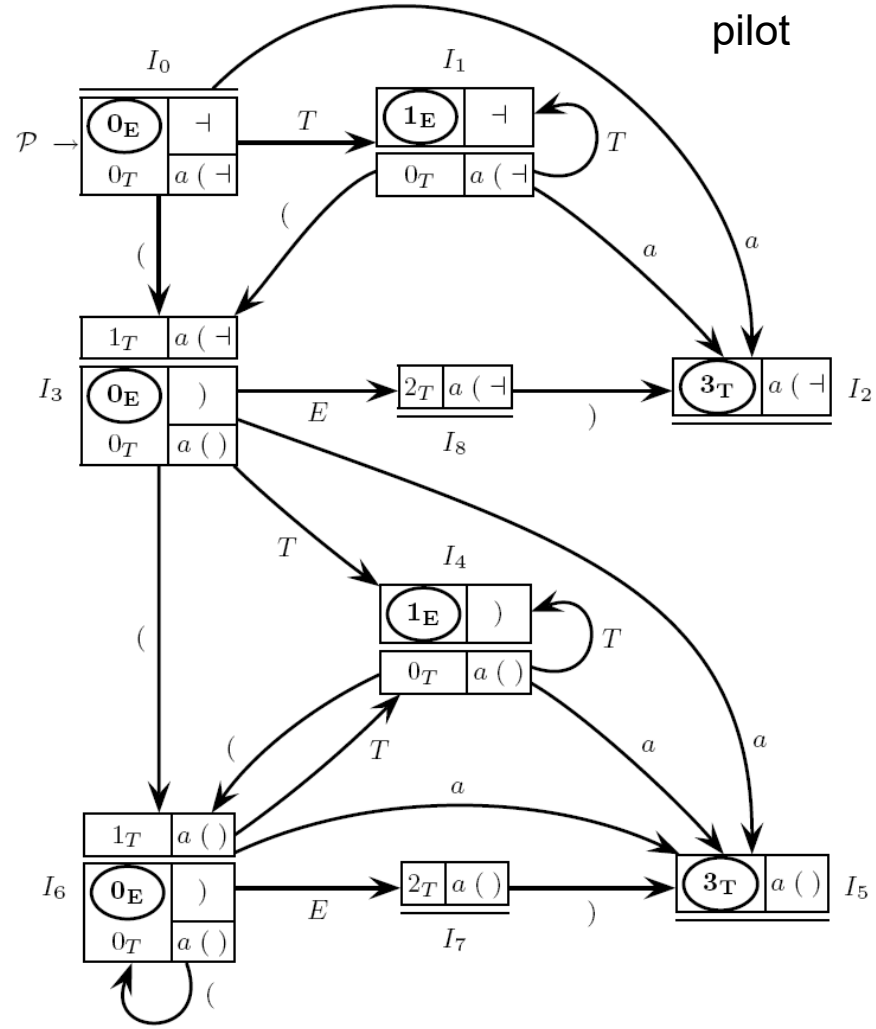
∃ a reduction item with look-ahead that overlaps with the terminal symbols on the outgoing arcs
⇒ *PDA* is unable to choose shift or reduction

reduce-reduce conflict

∃ two or more reduction items with look-ahead that overlap ⇒ *PDA* is unable to choose which reduction

m-state	*reduce-shift conflict*	*reduce-reduce conflict*
I_0	No: the reduction candidate is $\langle 0_E, \dashv\rangle$; the characters scanned are a and (, disjoint from $\{\dashv\}$	No: only one reduction candidate
I_1	No: the reduction candidate is $\langle 1_E, \dashv\rangle$; the characters scanned are a and (	No: only one reduction candidate
I_2	No: only one candidate which is a pure reduction	No: only one reduction candidate
I_3	No: the reduction candidate is $\langle 0_E,)\rangle$; the characters scanned are a and (	No: only one reduction candidate
I_4	No: the reduction candidate is $\langle 1_E,)\rangle$; the characters scanned are a and (	No: only one reduction candidate
I_5	No: only one candidate which is a pure reduction	No: only one reduction candidate
I_6	No: the reduction candidate is $\langle 0_E,)\rangle$; the characters scanned are a and (	No: only one reduction candidate
I_7	No: pure shift m-state	No: pure shift m-state
I_8	No: pure shift m-state	No: pure shift m-state

pilot



CONDITION *ELR* (1) – PART 2 – no transition has any *convergence conflict*

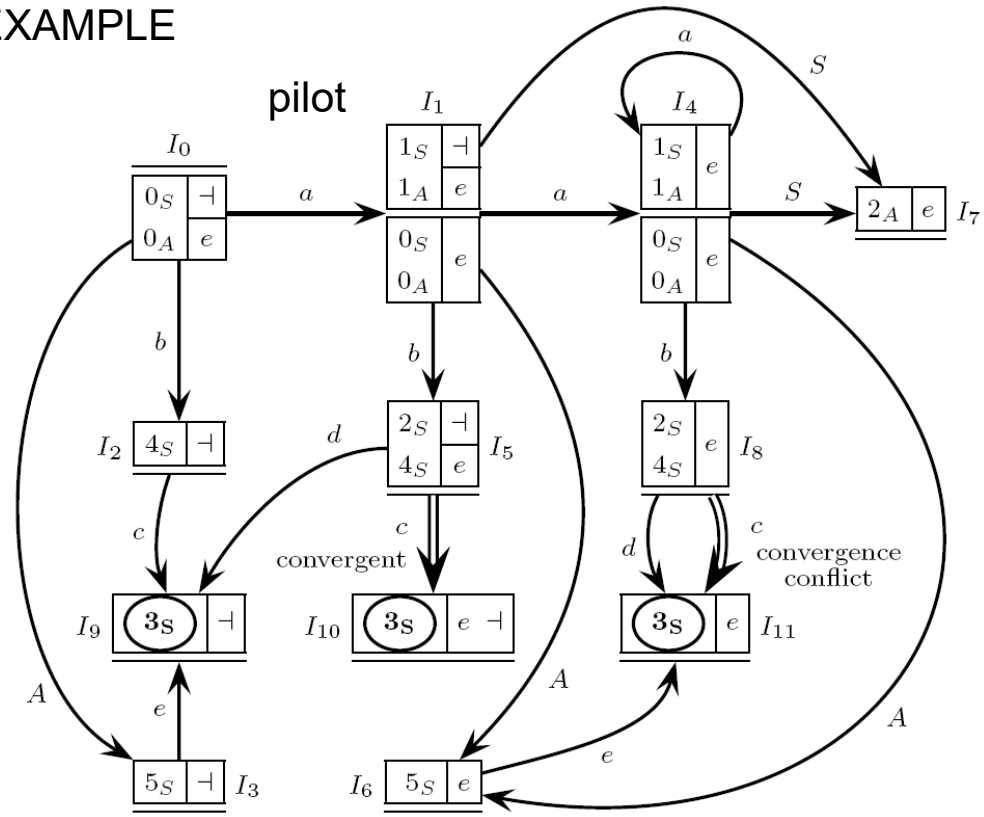
A m-state has a *multiple transition* if it contains two (or more) items $\langle\, p,\ \pi \,\rangle$ and $\langle\, r,\ \rho \,\rangle$ such that their two (or more) next states $\delta(p, X)$ and $\delta(r, X)$ are defined for a symbol X (terminal or non)

A multiple transition (see before) is *convergent* if $\delta(p, X) = \delta(r, X)$

A (multiple) convergent transition has a *conflict* if $\pi \cap \rho \neq \varnothing$

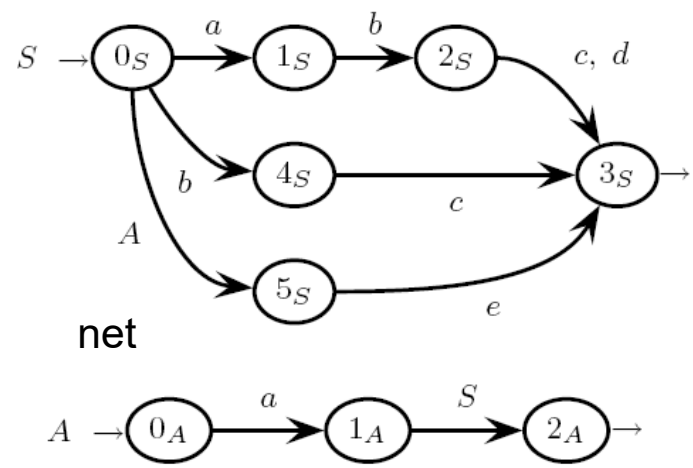
Two paths merge into one state and the *PDA* is unable to choose one reduction

EXAMPLE



$S \to a\,b\,(\,c \mid d\,) \mid b\,c \mid A\,e$

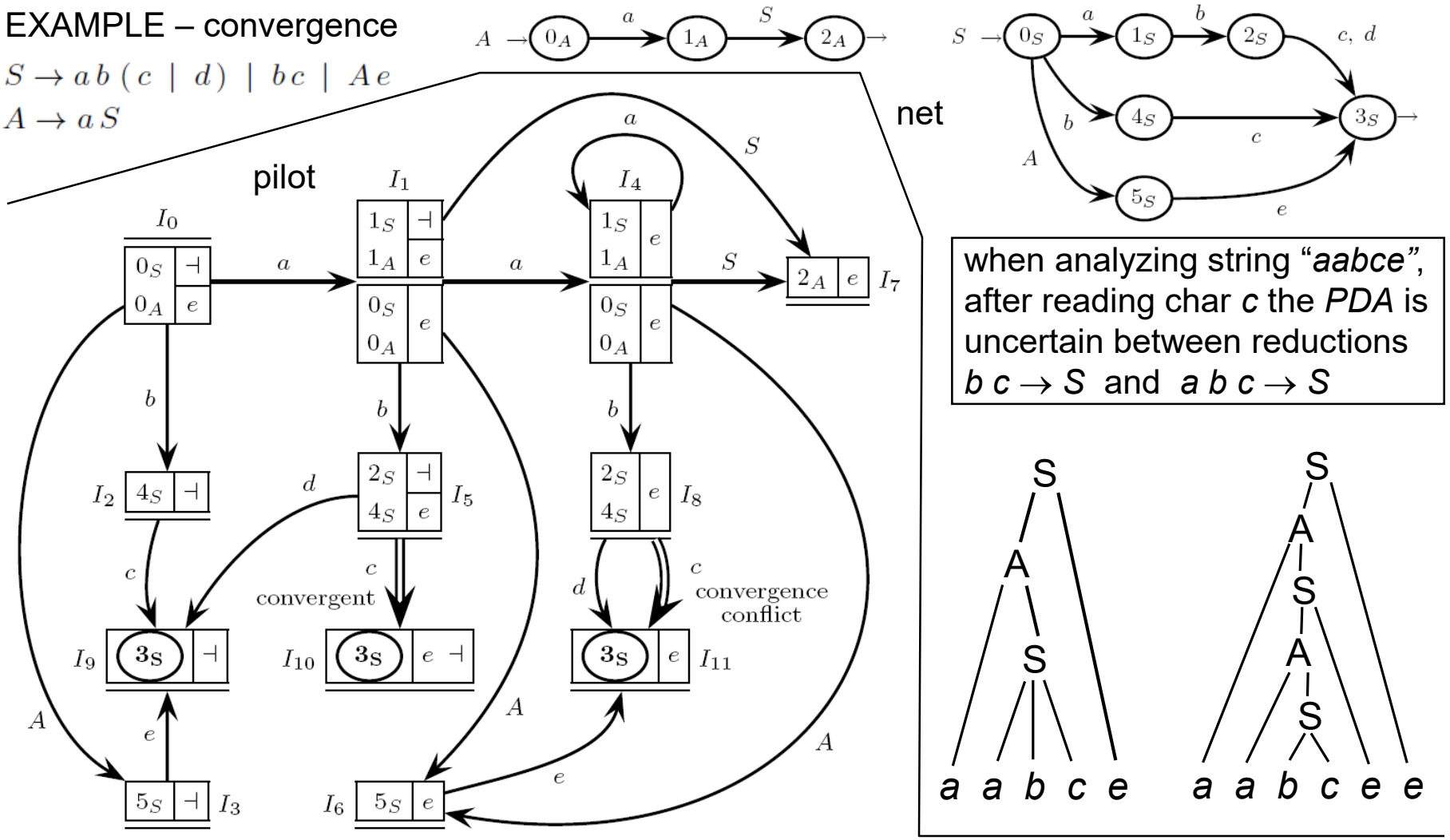
$A \to a\,S$



EXAMPLE – convergence

$S \to a\,b\,(\,c \mid d\,) \mid b\,c \mid A\,e$

$A \to a\,S$



when analyzing string "*aabce*", after reading char *c* the *PDA* is uncertain between reductions $b\,c \to S$ and $a\,b\,c \to S$

There are two net computations that can generate the string "*a a b c*", and both ones go into the same m-state with the same look-ahead

$$0_S \xrightarrow{\varepsilon} 0_A \xrightarrow{a} 1_A \xrightarrow{\varepsilon} 0_S \xrightarrow{a} 1_S \xrightarrow{b} 2_S \xrightarrow{c} 3_S \qquad 0_S \xrightarrow{\varepsilon} 0_A \xrightarrow{a} 1_A \xrightarrow{\varepsilon} 0_S \xrightarrow{\varepsilon} 0_A \xrightarrow{a} 1_A \xrightarrow{\varepsilon} 0_S \xrightarrow{b} 4_S \xrightarrow{c} 3_S$$

HOW THE PDA WORKS UNDER THE CONTROL OF THE PILOT GRAPH

The *PDA* scans a string and executes a sequence of shift and reduction moves

The *PDA* pushes groups of items and starts from those in the initial pilot m-state

Each m-state item becomes a 3-tuple by adding to it a backward-directed pointer that helps to reconstruct the different analysis threads constructed in parallel

The *PDA* decides whether to scan or reduce basing on the look-ahead in the pilot

If the condition *ELR* (1) is satisfied (all parts), then the *PDA* is deterministic

CRUCIAL ASPECT – LENGTH OF THE REDUCTION HANDLE TO BE POPPED

The length is not fixed, as a rule right part may contain regular operators like $*$ or $+$
$\Rightarrow$ a rule may generate phrases of unbounded length

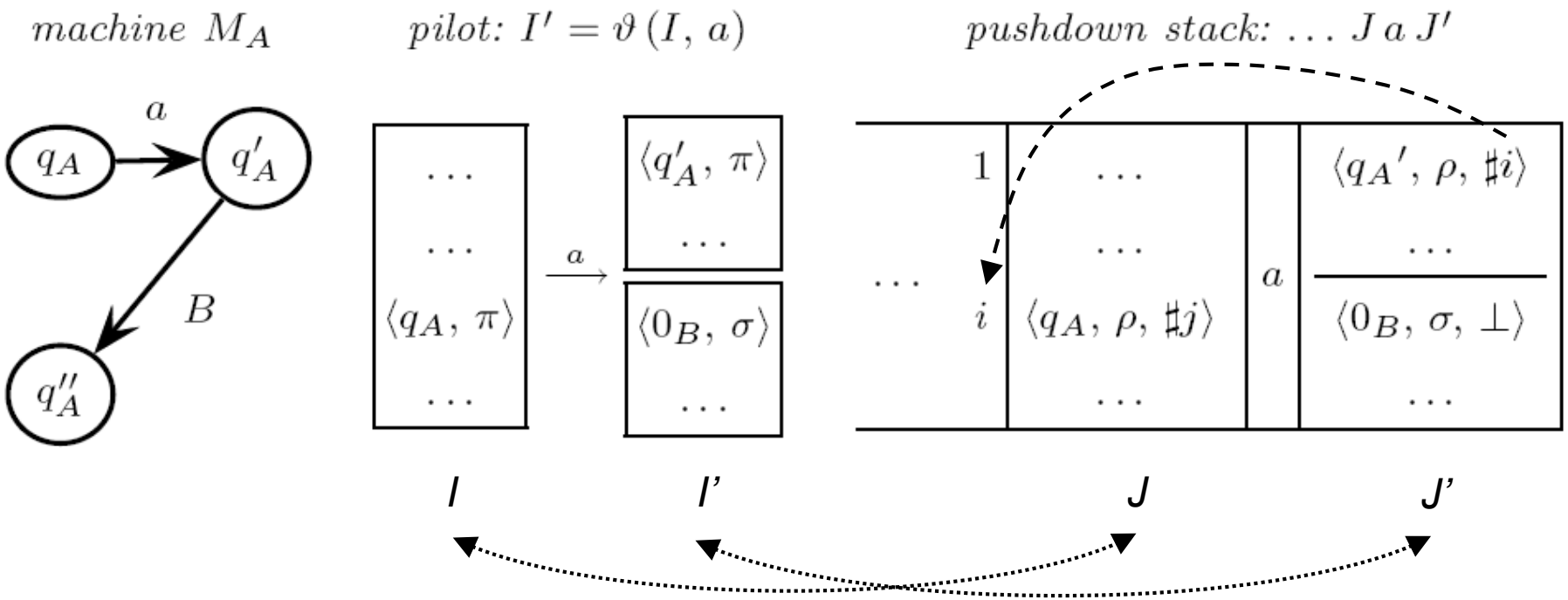
The reduction handle length is determined at reduction time by using the pointers

The pointer chain is followed backwards unto where the analysis thread started

A pointer value "$\perp$" identifies a thread start point and all the closure items have a pointer value "$\perp$", so these items are the start points of new threads

A 3-tuple with pointer $\neq \perp$ continues an already started thread; the pointers $\neq \perp$ are displayed as *#i*; *a* pointer value *#i* means that the pointer is targeted to the *i*-th item (from top to bottom) in the previous stack element

EXPLANATION OF THE SHIFT MOVE – a correspondence at three levels



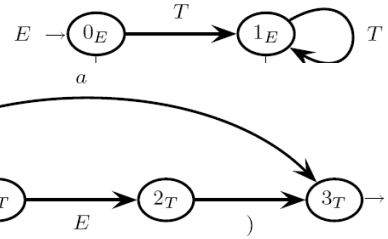
The stack m-states *J*, *J'* are similar to the corresponding pilot m-states *I*, *I'*, but their items have a backward-directed pointer

NOTE – the two look-aheads σ and ρ may be different notwithstanding they appear in two items with the same state, because the stack item in *J* may result from splitting the pilot item in *I* (this event may happen only if the transition is convergent – later an example)

RUNNING EXAMPLE – analysis trace of string " (( ) a) " 1/2 (CONTINUES)

For legibility we do not show the pushed input symbols and the item look-ahead symbols

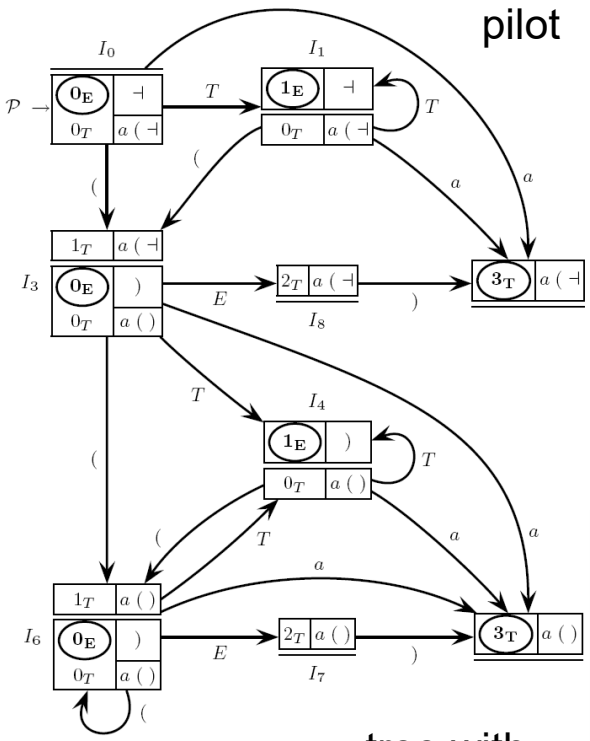
$E \to T^*$



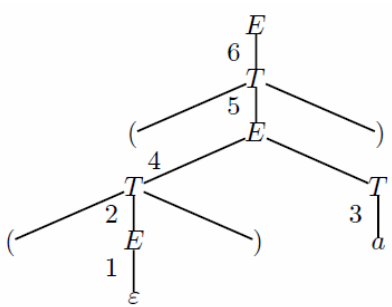
net

$T \to$ '(' E ')' | a

pilot



stack base	string to be parsed (with end-marker) and stack contents 1	2	3	4	5		effect after
	(	(	)	a	)	⊣	initialisation of the stack
	(	(	)	a	)	⊣	shift on (
	(	(	)	a	)	⊣	shift on (
	(	(E	)	a	)	⊣	1 reduction $\varepsilon \rightsquigarrow E$ and shift on E
	(	(E	)	a	)	⊣	shift on)
	(	T		a	)	⊣	2 reduction $(E) \rightsquigarrow T$ and shift on T



tree with reductions

MEANING OF THE ARROWS

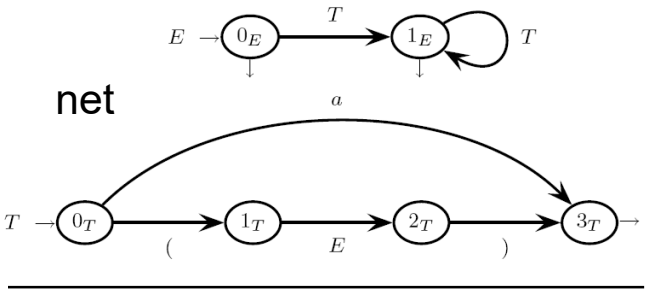
- - - - → *shift*

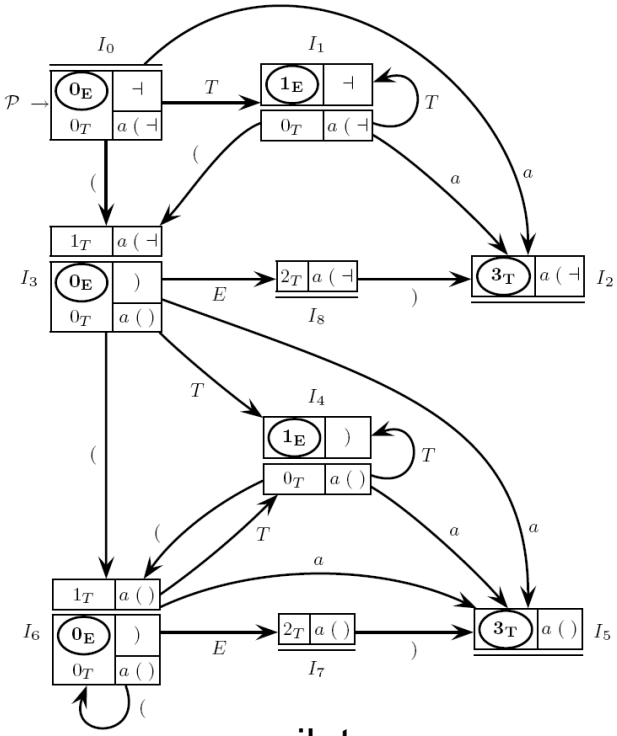
← *reduction* (*handle*)

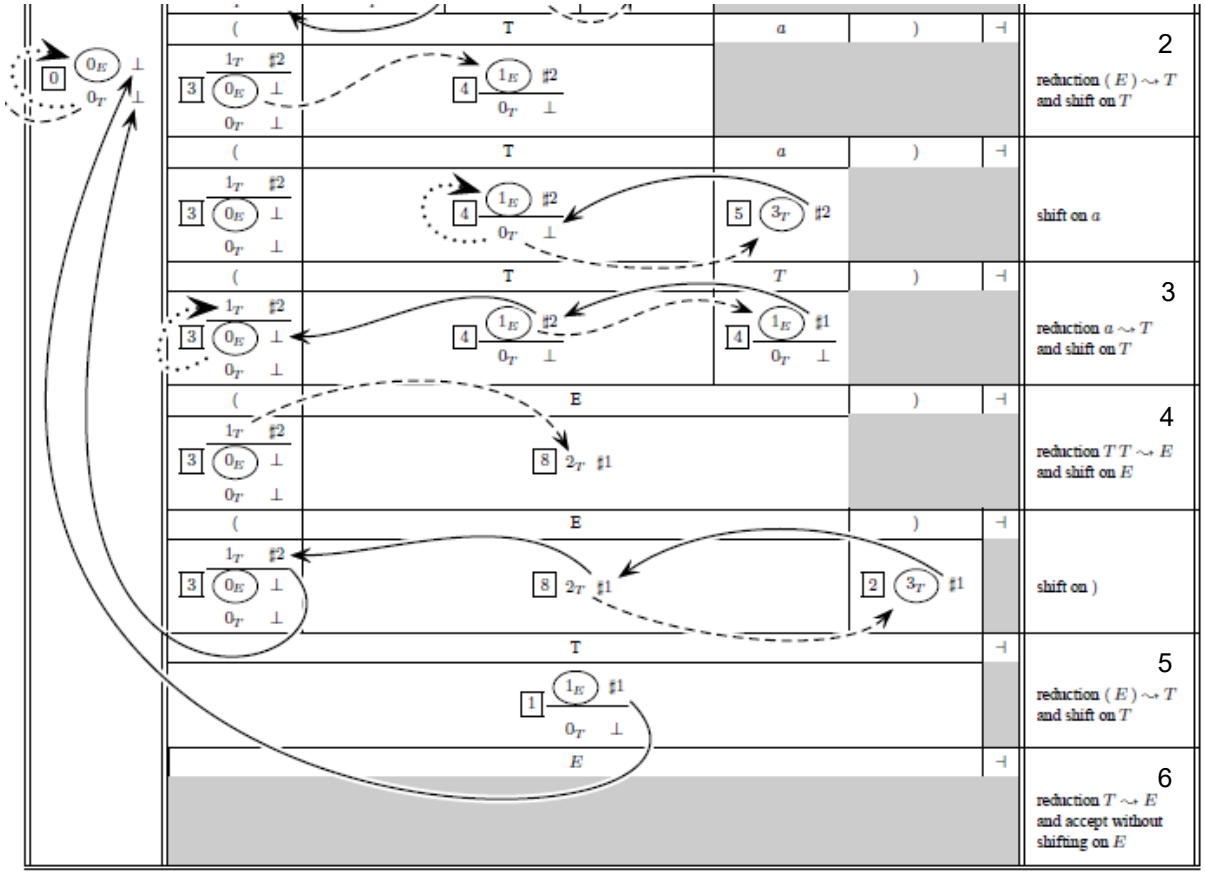
closure

k identifies the m-state

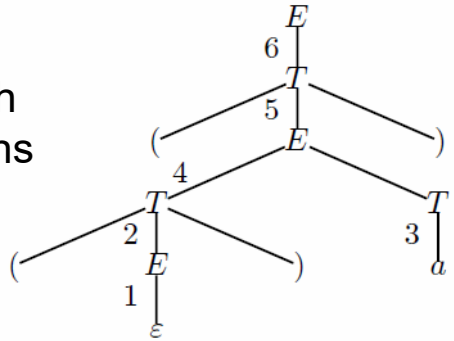
RUNNING EXAMPLE – analysis trace of string “ (( ) a) ” 2/2 (END)



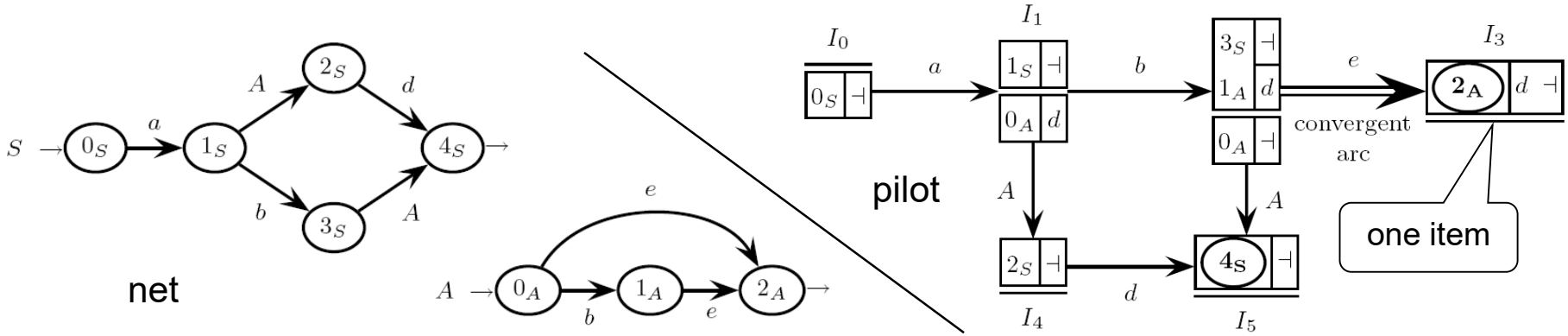




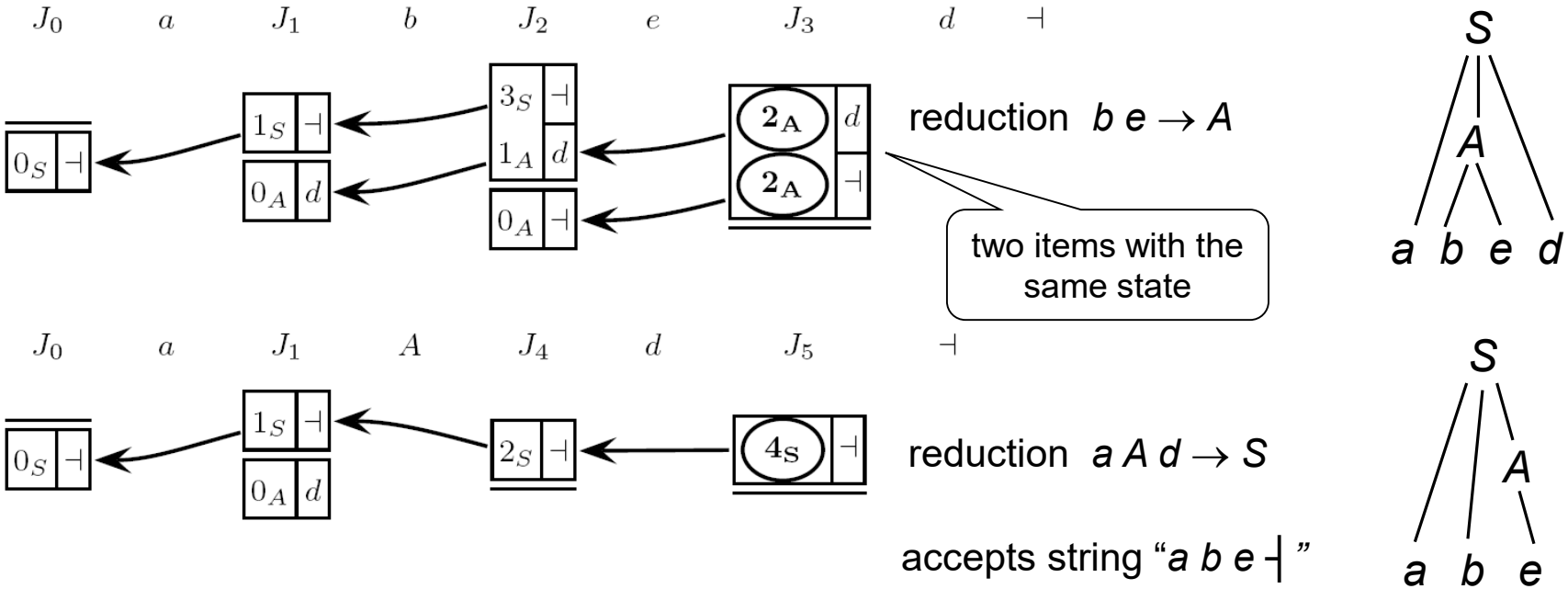
tree with reductions



EXAMPLE – if the pilot has a convergent transition, in the *PDA* stack there may be an element that has two or more items with the same state



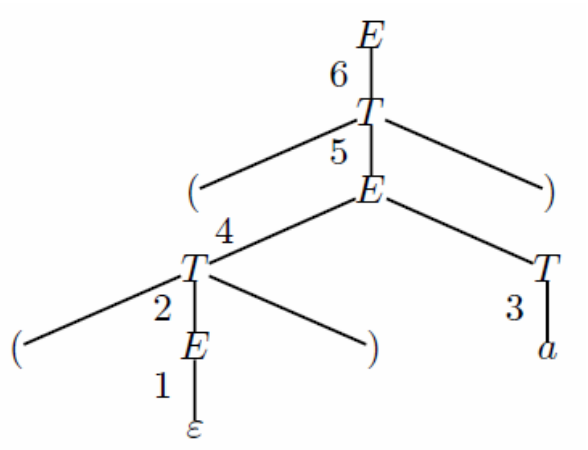
The analysis of the string "*a b e d* ┤" leads to J_3 with item in the same state 2_A; the pointers are not displayed as #*i*, but as arrows; to choose the correct reduction, use the look-ahead



OBSERVATION – what does the pilot recognize ? (if all the m-states are final)

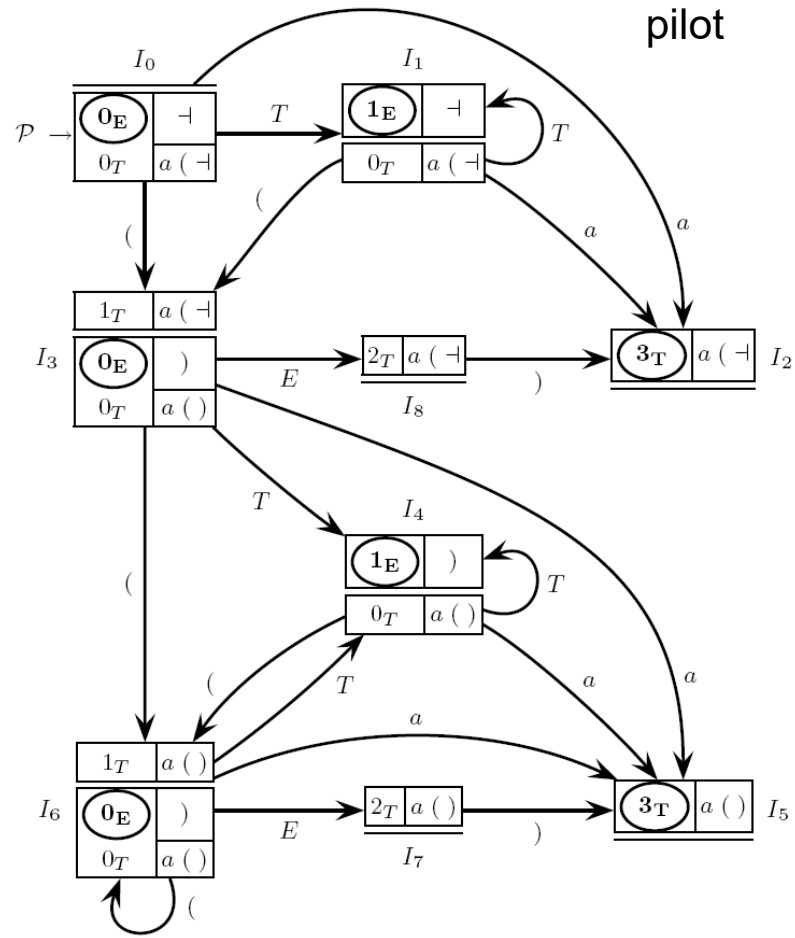
It recognizes all the strings that come from scan or reduction and appear in the stack

Such strings are called *viable prefixes* (or *handle prefixes*)



For the sample string " (( ) a) "
the viable prefixes are

(, ((, ((*E*, ((*E*), (*T*, (*Ta*, (*TT*, (*E*, (*E*), *T*



COMPUTATIONAL COMPLEXITY OF THE ANALYSIS

When analyzing a string x of length $n = |x|$, the number of elements in the stack is $\leq n + 1$

To count the number of PDA moves, consider these contributes

$n_T =$ # terminal shifts $n_N =$ # non-terminal shifts $n_R =$ # reductions

Obviously it holds $n_T = n$ and $n_N = n_R$, as for each reduction there is one non-terminal shift

$\Rightarrow$ the total number of *PDA* moves is

$$n_T + n_N + n_R = n + 2 \times n_R$$

Furthermore

- number of reductions with one or more terminals, e.g., $A \rightarrow a$, $A \rightarrow a\ B$, $A \rightarrow B\ a\ C$, is $\leq n$
- number of reductions of type $A \rightarrow \varepsilon$, i.e., null, or $A \rightarrow B$, i.e., copy, is linearly bounded by n
- number of reductions without any terminals, e.g., $A \rightarrow B\ C$, is linearly bounded by n

The analysis time complexity is $O(n) \leq k \times n + c$ for some integer constants k and c

The analysis space complexity is the max stack size, which is $n_T + n_N + 1 \leq k \times n + c$

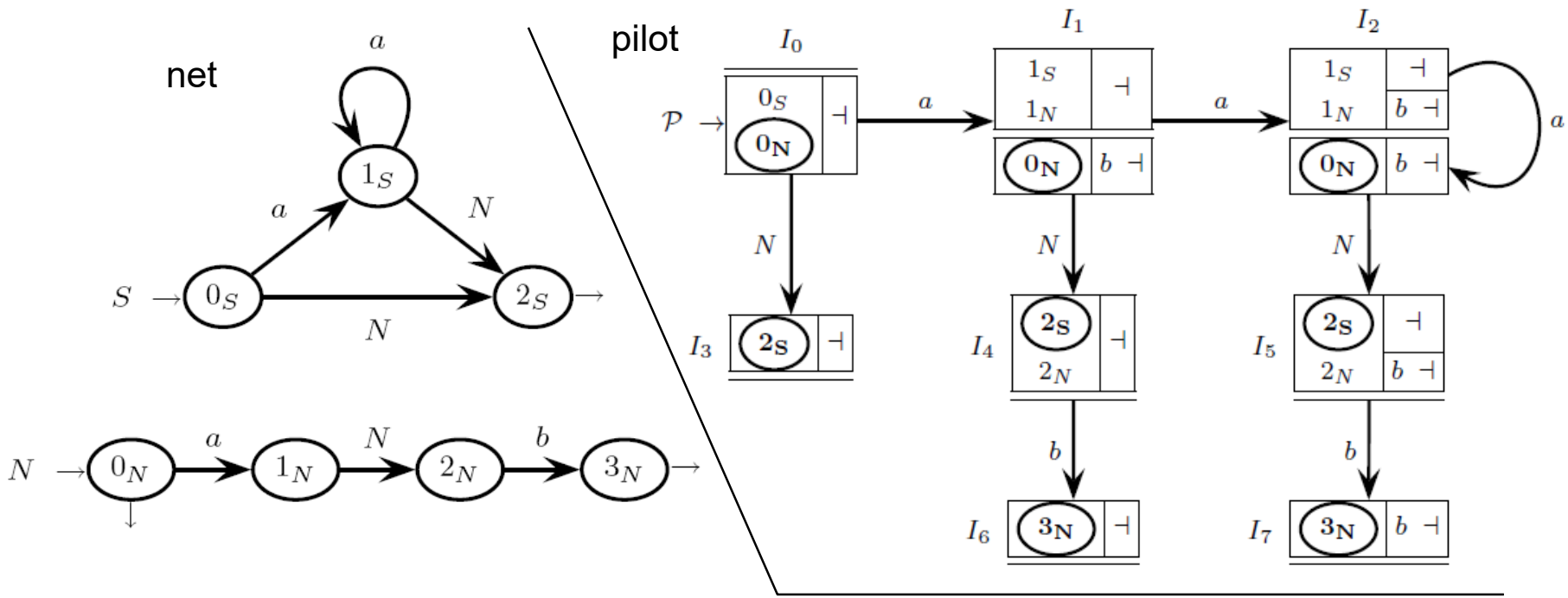
(space complexity is always upper-bounded by time complexity)

SAMPLE INTERESTING GRAMMARS AND THEIR PILOT GRAPHS

A GRAMMAR WITH TWO ITEMS IN THE M-STATE BASES

$$\begin{cases} S \to a^* \; N \\ N \to a \; N \; b \;\; | \;\; \varepsilon \end{cases}$$

It generates the deterministic language { $a^n \, b^m$ | $n \geq m \geq 0$ }

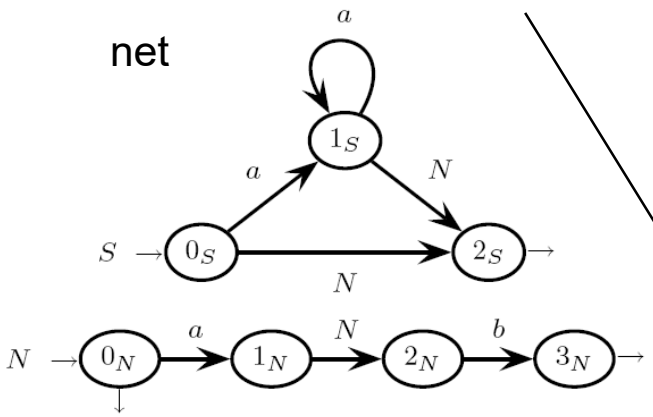


Two or more items in the m-state bases ⇒ the *PDA* carries on two or more parallel analysis threads, though there is only one final reduction because the condition *ELR* (1) is satisfied

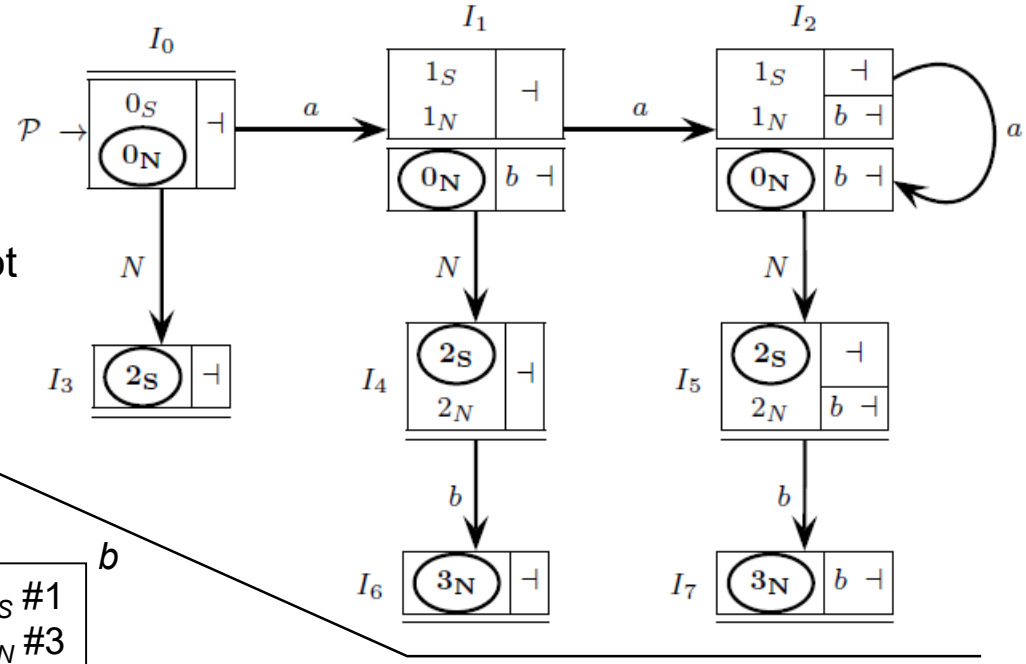
Try to simulate the analysis of a few sample strings: ε, *a a*, *a a b*, *a b*, *a b b*

EXAMPLE – analysis of "*a a a b* ⊣ "
uses all the net arcs

net



pilot



0	0_S ⊥ 0_N ⊥	a	1	1_S #1 1_N #2 0_N ⊥	a	2	1_S #1 1_N #3 0_N ⊥	a	2	1_S #1 1_N #3 0_N ⊥	b		

reduce $\varepsilon \to N$

0	0_S ⊥ 0_N ⊥	a	1	1_S #1 1_N #2 0_N ⊥	a	2	1_S #1 1_N #3 0_N ⊥	a	2	1_S #1 1_N #3 0_N ⊥	N	5	2_S #1 2_N #2	b	7	3_N #2

reduce $a\ N\ b \to N$

PDA stack simulations

0	0_S ⊥ 0_N ⊥	a	1	1_S #1 1_N #2 0_N ⊥	a	2	1_S #1 1_N #3 0_N ⊥	N	5	2_S #1 2_N #2

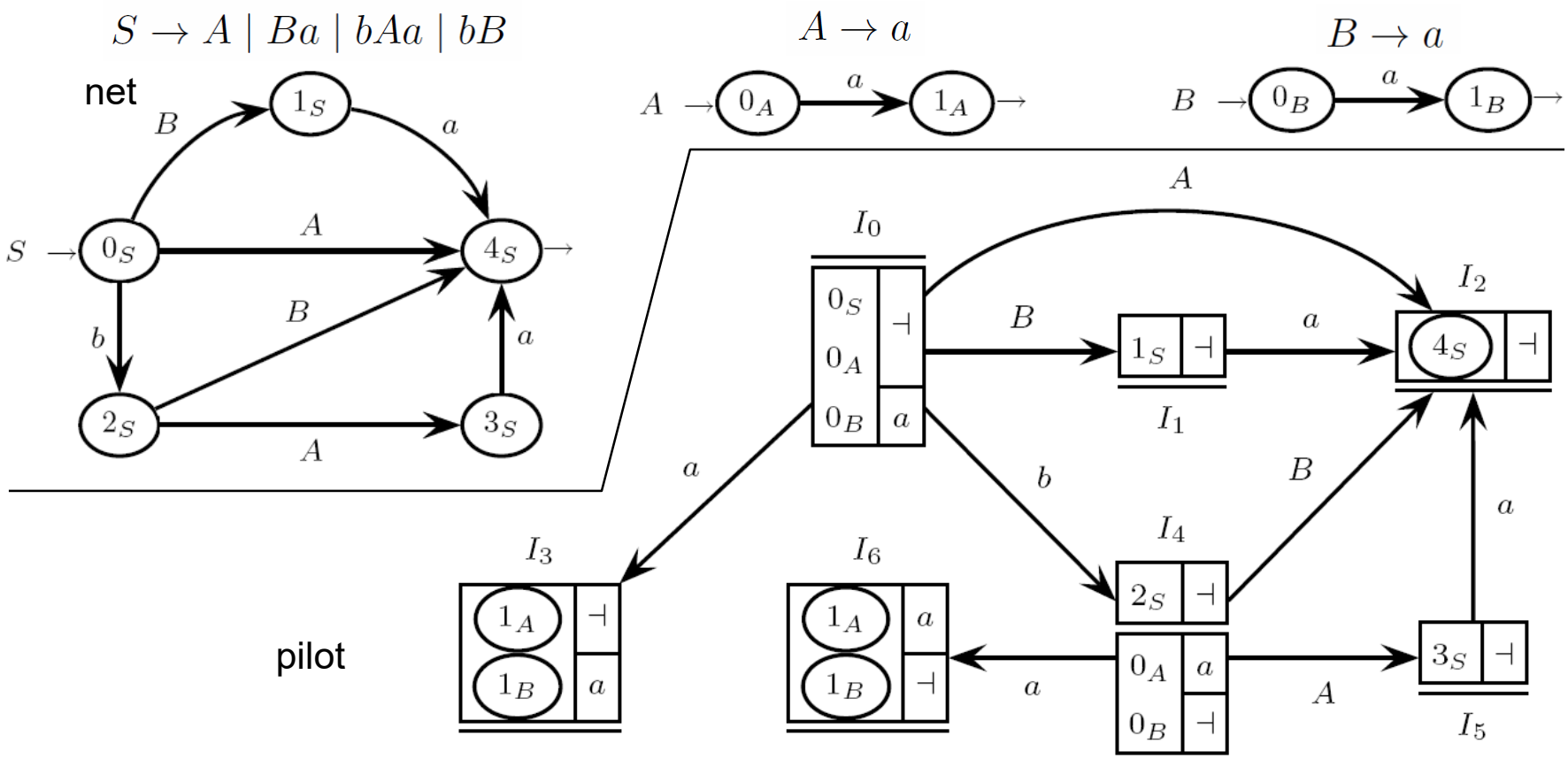
reduce $a\ a\ N \to S$

0	0_S ⊥ 0_N ⊥

accept

A GRAMMAR THAT HAS TWO REDUCTIONS IN A M-STATE
yet that does not have any reduce-reduce conflicts

Finite language (grammar not recursive) $L = \{\ a,\ a\,a,\ b\,a\,a,\ b\,a\ \}$



The m-states that potentially violate the *ELR* (1) condition are I_3 and I_6

Yet none of them has outgoing arcs, so there are not any shift-reduce conflicts

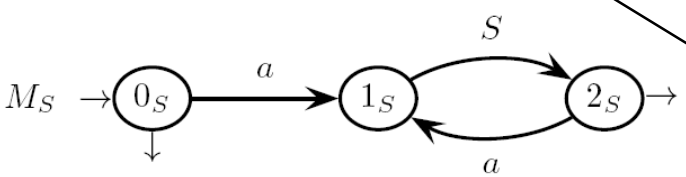
In I_3 and I_6 there are two reduction items with disjoint look-ahead $\Rightarrow$ no reduce-reduce conflict

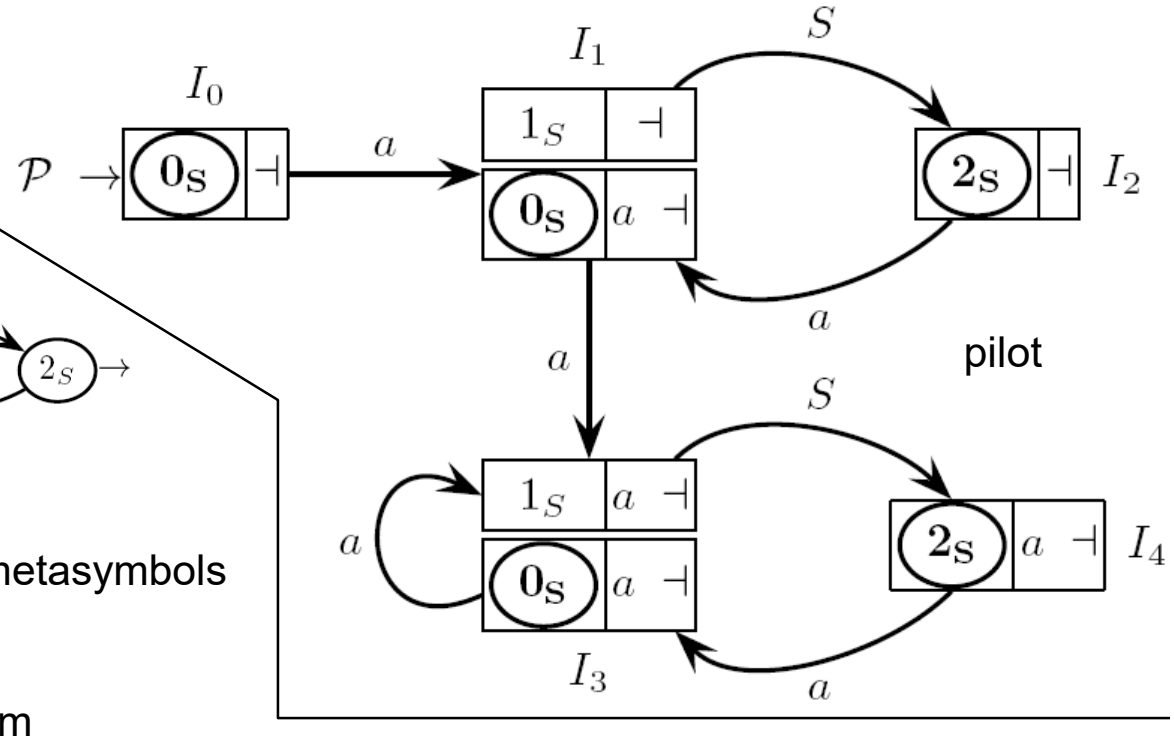
A GRAMMAR THAT VIOLATES THE CONDITION *ELR* (1) AS IT IS AMBIGUOUS

highly ambiguous rule

$S \rightarrow (a\,S)^*$

net





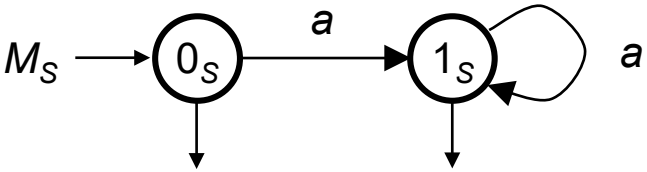
$L = a^*$ brackets "(", ")" are metasymbols

Various violations of determinism

- there are shift-reduce conflicts in the m-states I_1, I_3 and I_4
- char *a* is in the look-ahead of the reduction items 0_S and 2_S, as well as on an outgoing arc

A POSSIBLE REMEDY

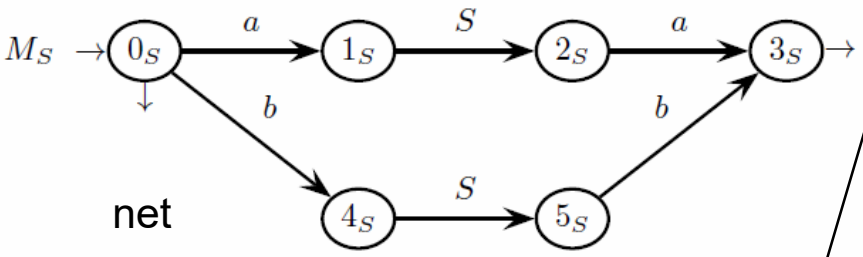
The language is $L = a^*$ and is regular. It is generated by grammar $S \rightarrow a^*$, which has a net like the one on the right that satisfies the condition *ELR* (1) (check it)



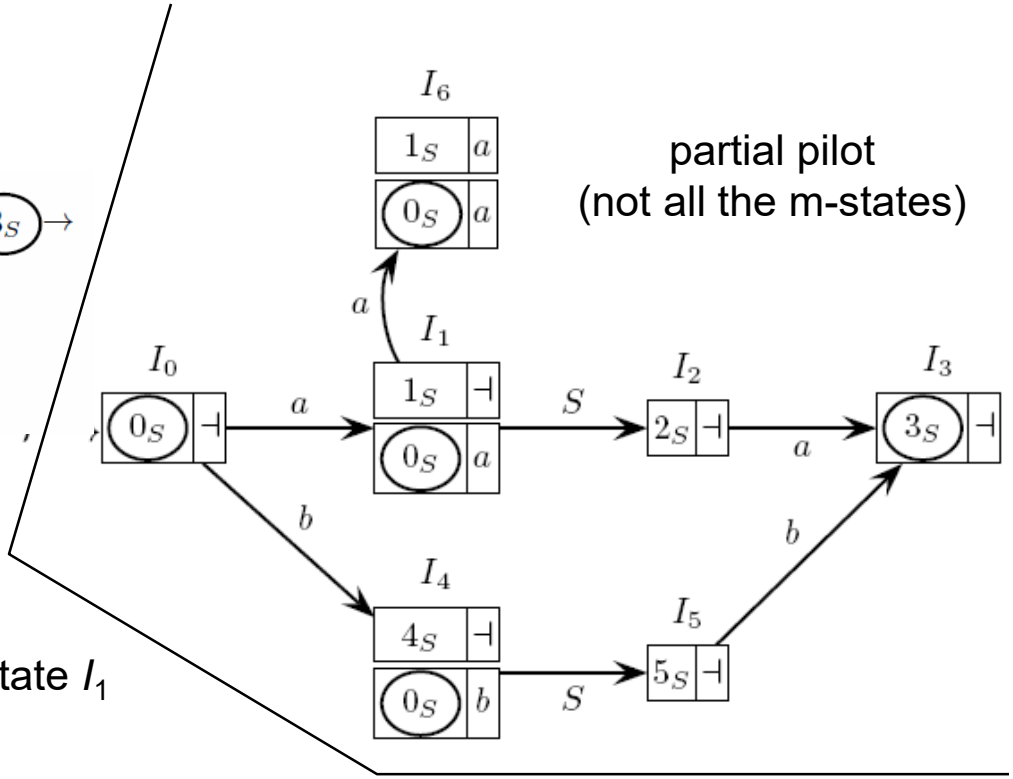
A GRAMMAR THAT VIOLATES THE CONDITION *ELR* (1) AS IT IS NON-DETERMINISTIC

palindrome

$$S \rightarrow aSa \mid bSb \mid \varepsilon$$



net



The language strings do not have a centre deterministically identifiable

There is a shift-reduce conflict in the m-state I_1 (and in I_4 as well though here not drawn)

Intuitively, after shifting a char *a* the *PDA* should (but is unable to) decide whether to

- *reduce*, if it has reached the palindrome centre
- *shift*, for the opposite reason (not in the centre)

Yet the *PDA* cannot know which case holds: the language is intrinsically non-deterministic

Such a language does not have any *ELR* (1) grammar, no matter how many rules are used

HOW TO IMPLEMENT THE *PDA* BY A VECTORED STACK

In an implementation of the *PDA* with some programming language, e.g., C, we can always mine the stack elements underneath the top one and directly look deep inside the stack

The third field of a stack item can be an integer that directly points back to the position of the stack element where the analysis thread begins

Actually the analyzer is no longer a true *PDA* as the stack alphabet becomes infinite

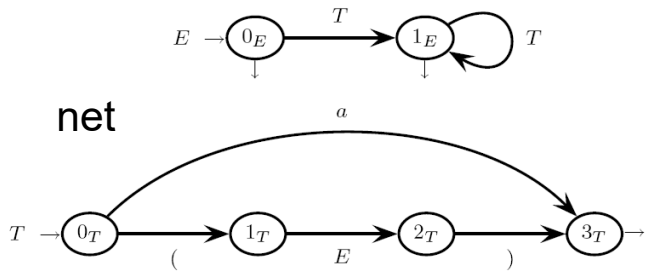
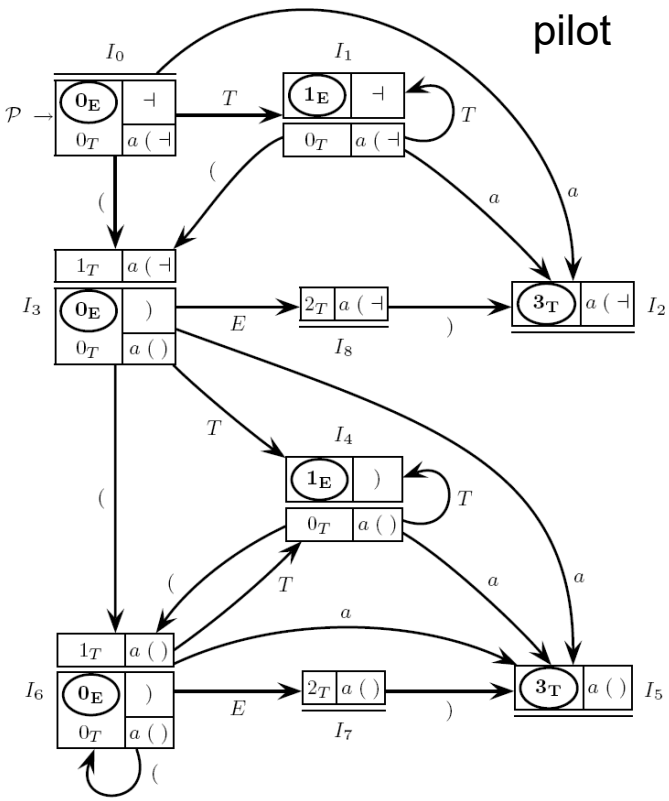
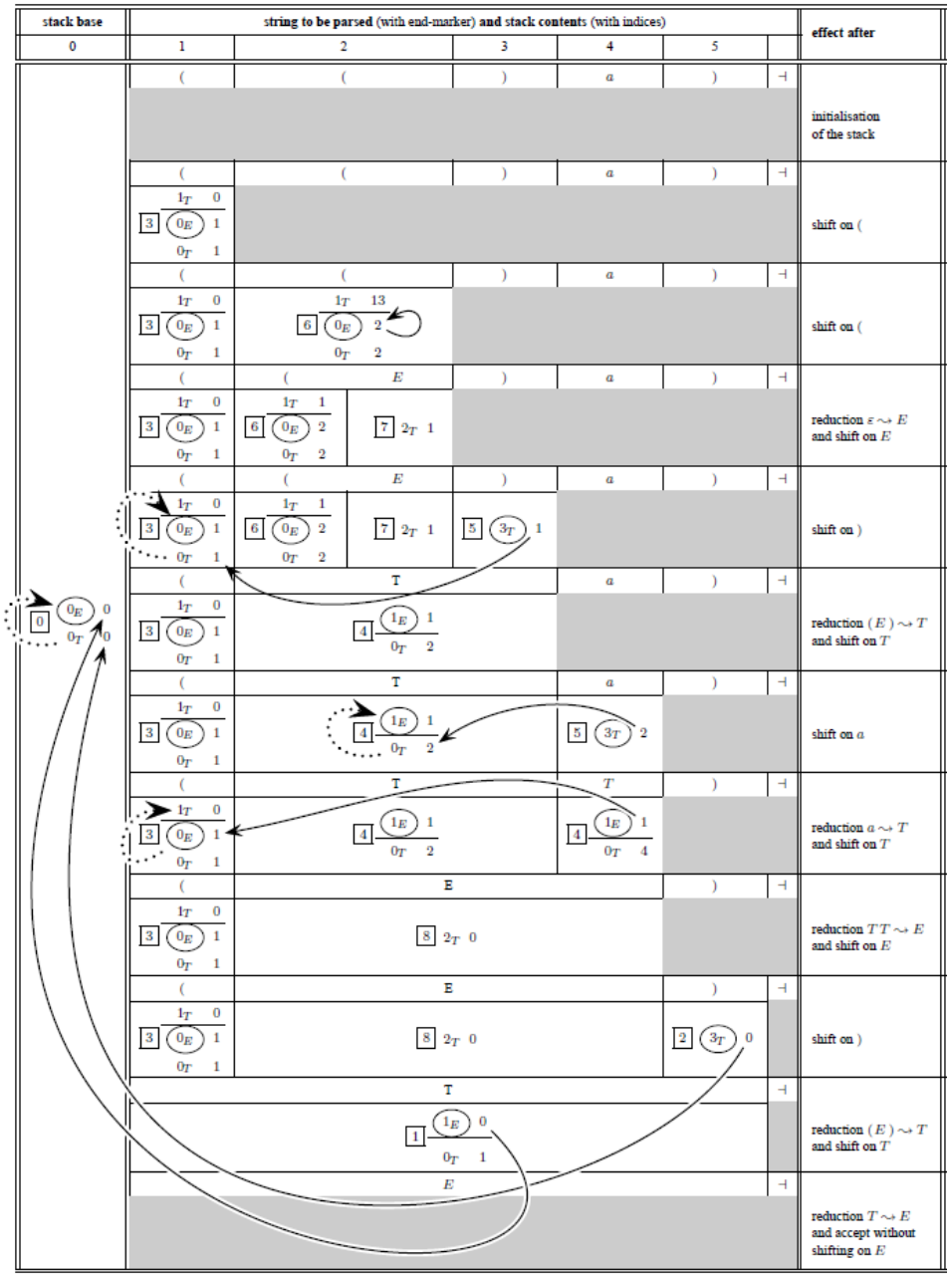
Such a variation is possible in every analyzer of practical interest and is not costly

In a closure item, write the current stack element index instead of the initial value "$\perp$"

In a base item, copy the same index value as that in the item before

So the *PDA* does not scan back the reduction handle and goes directly to the origin

IT IS SIMILAR TO THE EARLEY ALGORITHM (see later …)

RUNNING EXAMPLE – vectored stack *PDA*

Analysis trace of string " (( ) a) "